Guía de operación del gemelo WMS

wms-twin · PrismaTec-Core

Misma guía que doc/README.md — primera persona, operativa

Escribo este PDF para alinear el material imprimible con la guía viva en Markdown. Cualquiera del equipo debe poder arrancar, probar y ampliar el gemelo sin perderse.

Escribo esta guía en primera persona para que cualquiera del equipo pueda entender, arrancar, probar y ampliar el gemelo sin perderse. Es la misma historia que el PDF Guia_Completa_WMS_Twin_PrismaTec.pdf que acompaña esta carpeta: aquí en Markdown, allí listo para compartir o imprimir.

No es un tratado académico. Es el manual que yo usaría si mañana tuviera que volver a este código después de meses.

1. Qué es esto y qué no es

Mantengo el backend Flask original del monorepo sin tocar. El gemelo vive solo bajo wms-twin/.

El gemelo es un servidor en Go que habla el mismo idioma HTTP (/api/...) que ya usan el admin web y el mobile. La idea es poder apuntar los clientes al gemelo en laboratorio y, cuando el nivel de paridad y la operación lo permitan, usarlo como alternativa al Flask.

Por debajo no guardo el negocio en PostgreSQL como el original. Uso PrismaTec-Core: organismos, ledger de inventario, identidad de nodo y, si aplica, malla entre nodos.

Qué sí puedo afirmar hoy

• Arranco el gemelo en lab y responde health, login y flujos de almacén básicos.

• El admin puede autenticarse con cookies + CSRF; el mobile con Bearer JWT.

• La batería de scripts de esta carpeta scripts/ valida esos caminos.

Qué no afirmo todavía

• Sustituir Flask en producción sin E2E visual completo, sin paridad profunda de todos los módulos y sin plan de datos/rollback.

2. Dónde está cada cosa (mapa del directorio)

```
wms-twin/
  cmd/wms-twin/          → punto de entrada del binario
  internal/
    features/            → una carpeta por capacidad de negocio (feature-first)
    domain/              → reglas y tipos más “puros”
    ports/               → contratos (interfaces) que las features necesitan
    infrastructure/      → adaptadores de persistencia sobre PrismaTec
    corestore/           → acceso a organismos del Core
    httpapi/             → capa HTTP (rutas /api compatibles con Sentry)
    security/            → JWT, CSRF, tokens de integración
    runtime/ mesh/ …     → arranque del nodo y red
  scripts/               → demos y E2E (curl, cookies, Bearer)
  demo/                  → panel HTML de laboratorio
  proxy/                 → ejemplos de proxy
  doc/                   → ESTA guía, el PDF y archive/ histórico
  README.md              → acceso rápido a esta guía
```

Si busco “dónde se crea un almacén”, empiezo en httpapi, sigo a features/catalog y termino en corestore / PrismaTec.

3. Cómo se enlazan las capas (Clean Architecture en la práctica)

Pienso el flujo siempre igual:

• Llega una petición a /api/....

• httpapi comprueba sesión (cookie o Bearer) y CSRF si aplica.

• Llama a un caso de uso en features/.

• Ese caso de uso habla con un repositorio (port), no con el detalle del disco.

• La infraestructura (corestore, features/admindata, etc.) escribe o lee organismos PrismaTec-Core.

Así, cuando cambio persistencia o completo un módulo que antes estaba a medias, no reescribo los controladores HTTP: cambio la implementación del repositorio o quito datos semilla de la capa de datos.

```
Cliente (admin o mobile)
```

```
→ httpapi
    → features/...
        → ports (contratos)
            → infrastructure / corestore
                → PrismaTec-Core (nodo + organismos + ledger)
```

4. Features: qué hace cada paquete

Carpeta en features/	Para qué la uso
identity	Usuarios, login, bloqueos
catalog	Almacenes, zonas, bins, ítems, vendors
receiving	Recepción y putaway
picking	Oleadas y tareas de picking
fulfillment	Empaque y despacho
salesorders / purchaseorders	Órdenes de venta y compra
inventoryledger	Movimientos de stock con idempotencia
pos / dockd / rma	Punto de venta, dock, devoluciones
admindata	Datos de admin aún en migración; repositorio estilo BD
events / webhooks / inbound / snapshot	Integraciones y eventos

ports/AdminResourceRepository es el contrato que cumple admindata.Store: listar, obtener, guardar y borrar recursos de admin como si fueran filas, no JSON inventado en el controlador.

5. PrismaTec-Core en palabras simples

• Nodo: el proceso del gemelo “es” un nodo Core con carpeta de datos e identidad.

• Organismo: un registro de negocio (un almacén, un SO, un canal) con identificador y huella de contenido.

• corestore: mi puente Go para crear/leer/listar/borrar esos organismos.

• Ledger: el diario de movimientos de inventario (ajustar, mover, reservar) con claves de operación para no duplicar efectos.

• AIP: API propia del Core en otro puerto; el admin/mobile de Sentry no la necesitan para el día a día.

• Mesh: cuando hay varios nodos, permite difundir eventos; en lab local a menudo no la necesito.

No imito una blockchain de terceros: uso el modelo de organismos e integridad que ya trae PrismaTec-Core alineado a este WMS.

6. Admin y mobile: dos formas de sesión (y por qué)

Admin (navegador)

El admin no guarda el JWT en localStorage. El servidor pone:

• sentry_auth → cookie HttpOnly (JavaScript de la página no puede leer el token; reduce riesgo si hubiera XSS).

• sentry_csrf → cookie legible; el front copia el valor al header X-CSRF-Token en POST/PUT/PATCH/DELETE.

Así el navegador envía la sesión sola, y las escrituras exigen el token CSRF.

Mobile (app)

La app guarda el JWT en almacenamiento seguro del dispositivo y envía Authorization: Bearer ... en cada llamada. No usa el par de cookies de sesión web, así que el modelo CSRF del navegador no aplica igual.

El gemelo entiende los dos caminos. Por eso tengo scripts distintos de prueba.

7. Cómo lo arranco en mi máquina

```
cd wms-twin
./scripts/start-lab.sh
```

Usuarios de laboratorio habituales:

• operator / Operator123

• admin / Admin1234

API por defecto: http://127.0.0.1:5001

8. Cómo lo verifico (orden que uso)

```
# Núcleo de demo
./scripts/team-curl-demo.sh
```

Para abrir el admin en el navegador sin modificar el frontend, uso el proxy de lab:

```
# terminal A: gemelo
./scripts/start-lab.sh
```

Abro http://127.0.0.1:3080/ — el proxy manda /api al gemelo y el resto al Vite del admin.

9. Cómo añado una funcionalidad nueva (sin ensuciar)

• Decido a qué feature pertenece.

• Escribo el caso de uso en internal/features// (sin HTTP).

• Si necesito persistencia nueva, amplío el repositorio o un port en internal/ports y la implementación sobre corestore / PrismaTec.

• Expongo la ruta en httpapi lo más delgada posible: auth, parseo, llamada al caso de uso, respuesta.

• Registro la ruta en el mux de httpapi/server.go.

• Pruebo con go test y, si es visible al cliente, un curl o el script E2E que corresponda.

• Si ese endpoint vivía solo como dato semilla en admindata, quito la semilla cuando ya tenga dominio real.

Nunca invento el cuerpo JSON “a mano” dentro del handler para fingir una BD. Eso vive en la capa de datos.

10. Cómo sigo una dependencia cuando me pierdo

• Busco la ruta: rg 'HandleFunc("/api/" internal/httpapi

• Abro el método del Server y veo a qué feature llama.

• En la feature busco el repositorio o el ledger.

• Si es admin genérico: features/admindata y el kind wms_admin_....

```
rg -n "wms_admin_channel" internal/
rg -n "func \(s \*Server\) admin" internal/httpapi/
```

11. Estado de paridad (lectura honesta)

Objetivo	Estado
Demo lab / equipo con scripts	Listo para mostrar
Auth admin (cookies) y mobile (Bearer)	Validado con scripts E2E
Happy path almacén (recibir, putaway, listar SO…)	Cubierto en demo curl
Cada pantalla del admin/mobile en producción	Aún requiere E2E visual y cierre de dominio
Apagar Flask en producción	No hasta paridad operativa + datos + rollback

Detalle histórico y matrices antiguas: carpeta doc/archive/ (solo referencia; la guía viva es este README y el PDF).

12. PDF hermano

El archivo Guia_Completa_WMS_Twin_PrismaTec.pdf en este mismo directorio doc/ cuenta la misma infraestructura, con el mismo orden mental (componentes, capas, features, Core, auth, arranque, cómo extender, verificación). Si actualizo esta guía de forma sustancial, mantengo el PDF alineado.

13. Reglas que me imponen disciplina

- No modifico el frontend ni el backend Flask desde el trabajo del gemelo.
- Documentación del gemelo solo bajo wms-twin/doc/.
- Código del gemelo solo bajo wms-twin/.
- Preferir un repositorio / port antes que lógica de negocio en el handler HTTP.
- Cada cambio que toque auth o floor lo valido con los scripts de scripts/ antes de darlo por bueno.

Guía alineada con wms-twin/doc/README.md